

Last updated: February 7, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 6

A: Getting Started

Download [3000pc.zip](#), unpack, and run make to compile 3000pc-fifo and 3000pc-rendezvous.

Try to take this opportunity to read and understand the two programs line by line.

Note that these programs take 3 arguments each:

- The number of events to process
 - The number of events to produce before the producer sleeps for 1 second
 - The number of events to consume before the consumer sleeps for 1 second
1. Run both programs with the same arguments of your choice. Do this a few times with different arguments. Do they behave the same way? Do you notice any differences? (again, no “correct” answers but just play with them and document your observations)
In addition to other differences by design (e.g., the pipe’s buffer being much bigger than the 32 words of the shared memory), an important difference is that 3000pc-rendezvous’s event sequence is less deterministic (or you can say stable) than 3000pc-fifo. You can see this almost each time you run 3000pc-rendezvous. The order the producer and the consumer finish and whether they wait vary.
 2. Repeat the above experiment, this time running each program under strace (with the -f flag to trace children too). Do you notice any difference in the system calls each program makes? Remember to ignore the lines before main() that might be distracting, e.g., start looking around and after the clone() system call.
3000pc-fifo just reads/writes from/to the pipe descriptors to feed/consume words, so you see a lot of calls to read() and write() but with a simple pattern. By contrast, 3000pc-rendezvous involves multiple calls to futex(), which is the system-call implementation of those pthread mechanisms (you can ignore write() for printf()).
 3. You can see there is "#define QUEUESIZE 32" in both programs. Are they actually used? Why (not)?
Apparently, 3000pc-rendezvous relies on this value to determine the size of the shared memory maintained by ourselves. By contrast, 3000pc-fifo just makes use of the OS-maintained pipe via the pipe() system call and read()/write(), with no need for QUEUESIZE.

B: Producer/Consumer with Pipes

1. Examine the source code of 3000pc-fifo. Pay attention to the call to pipe(pipefd) on line 192 and make the connection with what has been discussed in the lecture (also look at the man page for pipe(2)). Explain the following:

- a. How does the consumer receive words to consume?
By reading from the read end of the pipe. As mentioned, the `pipe()` system call returns an array of two file descriptors, one for the read end and one for the write end.
 - b. How does the producer generate and send words to the consumer?
By writing to the write end of the pipe.
 - c. Why is the call to `srandom(time(NULL))` on line 169 necessary?
Otherwise, the generated sequence will always be the same if `random()` is not seeded properly (e.g., always seeded with the same value).
2. Replace the call to `srandom(time(NULL))` on line 169 with `srandom(42)`. What differences do you notice in `3000pc-fifo`'s behavior?
The same as above. `srandom(1)` (not seeded is equivalent to seeded with 1 or any constant value) vs. `srandom(42)`.

C: Producer/Consumer with Shared Memory

1. Examine the source code of `3000pc-rendezvous`. Explain the following:
 - a. What are a few ways that `3000pc-rendezvous` is different from `3000pc-fifo`?
You can have your own observations (e.g., IPC differences, with/without concurrency issues, determinism, etc.).
 - b. What does the call to `mmap()` on line 347 do (compared to our previous uses of `mmap()`)? How did you figure this out?
Refer to the summary of `mmap()` uses in the lecture. In this case, it's used to create a shared memory region between two processes. Check the man page of `mmap()`.
 - c. How does the producer notify the consumer that the queue is no longer empty?
See the additional explanation at the end.
 - d. How does the consumer notify the producer that the queue is no longer full?
See the additional explanation at the end.
2. What arguments can you provide to make the producer wait for the consumer? Hint: Check the size of the queue.
As long as the number of events is greater than the queue size and you slow down the consumer, the producer needs to wait.
3. What arguments can you provide to make the consumer wait for the producer? (You can disregard the wait at the very beginning as the queue is initially empty)
Different from the producer, the consumer relies on available items generated by the producer. So, as long as the producer is slower (e.g., by waiting more) than the consumer, the consumer needs to wait.
4. Another student tells you that the difference between processes and threads is that processes never share memory, while threads do. Is this statement correct or incorrect? How can the behavior of `3000pc-rendezvous` help justify your answer?
Apparently, we are using shared memory between processes.
5. Change the calls to `pthread_mutexattr_setpshared()` and `pthread_condattr_setpshared()` (lines 293 and 298) to take 0 instead of 1. How does the behavior of `3000pc-rendezvous` change? Does anything break?
Things will get stuck, because that makes the mutex and condition variable not accessible between processes (even if they share the memory where the mutex and variable reside). See

the man page.

Additional explanation for `3000pc-rendezvous`:

There are three mechanisms involved, 1) the per-entry semaphore (`sem_t lock`), which ensures that an entry is not manipulated by both parties at the same time. 2) the condition variable (`queue_nonempty/nonfull`), used to coordinate between the producer and the consumer so that they know they can stop waiting and work. 3) the mutex (`cond_mutex`), as is required by the condition variable.

There is also the fourth one, which is the Boolean variable `predicate_nonempty/nonfull` used in the predicate before each condition wait. Otherwise, you may notice a deadlock after certain number of events (usually large, e.g., 400,000). While there could be various factors causing this, one known issue is called spurious wakeup or lost wakeup. A brief explanation is: when a process yields the CPU to wait on something, it no longer runs. The OS/scheduler will wake it up when the condition is satisfied. However, waking per se might not be atomic or even reliable. So, by the time the waken process gets the chance to run, that condition may no longer hold (e.g., another process jumped in before and made a change). A predicate with a loop will ensure that the process will loop for one more iteration (hence waiting again) if spurious wakeup happens.